Twenty-Three Thousand Songs on a 2010 Arcade Machine

Characterising and extending the rhythm game EZ2AC FNEX for a community BMS library on its original hardware

ECas95

Independent research · October 2026 · Private technical report

Abstract. EZ2AC FNEX is the last release of a Korean arcade rhythm-game series whose arcade line has been dormant since 2020. It ships about 430 songs and was built for fixed arcade hardware. We describe how the game was characterised in enough depth to load a community library of 23,587 BMS songs into it, without modifying its files, on its original Windows XP cabinet: a 2010-era dual-core PC with 2 GB of RAM. We combine four decompilers, a separate byte-matching decompilation, run-time observation from inside the process, sweeps over all ~290,000 shipped data files, and measurements on the cabinet. Three results come out of this. First, a characterisation of the engine, its file formats and its timing model: judgement happens in chart ticks of 1/192 of a beat with a fixed five-tick latency compensation; each chart ships its own judgement windows (327 distinct sets); hold notes are graded twice but counted once; and the two scoring models are flat per-tier in the key modes and logarithmic in Ez2Catch. Second, a conversion from BMS to the game's format: a finer tick grid that removes a 4.17 ms mean displacement affecting 24% of notes, repair of malformed long notes, an AutoChart generator that learns lane usage and timing statistics from the game's own charts, and a note-based difficulty rating that spreads a library in which 46% of songs claimed the top level. Third, a performance study on the original hardware, which found and removed two freezes, of 0.7–1.7 s and 0.4–0.6 s, that no development PC could reproduce.

Keywords: reverse engineering, rhythm games, game preservation, file formats, timing analysis, BMS, legacy hardware, Windows XP.

1. Introduction

Arcade rhythm games are an awkward subject for preservation. Their software is tied to a cabinet, their song lists are fixed at release, and when a series ends, the machines remain in arcades and private collections with no new content and no maintainer. EZ2AC FNEX (Final EX), the last entry of a series started by Amuse World in 1999, is such a game. Its community, meanwhile, keeps a large library of charts in BMS, the open text-based chart format used by most community rhythm-game players.

This report describes the work needed to make the original game play that library on its original cabinet. The constraints were strict:

- the game's files are not modified;
- the hardware is a 2010-era PC running Windows XP;
- the result must feel like the original game: the same judgement, the same scoring, the same screens, with 55 times as many songs.

Our contributions are:

- a **methodology** that combines five independent lines of evidence and replaces reading with mechanical verification wherever possible (Section 3);

- a **characterisation** of the game's binary, engine, file formats, judgement timing, hold notes and scoring (Section 4);
- a **BMS-to-FNEX conversion** backed by measurements over the library: grid choice, long-note repair, lane folding, a learned chart generator and note-based rating (Section 5);
- a **performance study** on the original hardware (Section 6), and the lessons we draw from the whole effort (Section 7).

2. Background

2.1 The game and its platform

EZ2AC FNEX is a Windows program written in C++ against a small in-house engine. It renders 640×480 graphics with DirectDraw 7 and Direct3D 7, plays audio through DirectSound, and reads its controls through DirectInput and the cabinet's I/O board. Thirteen play modes range from 5-key layouts with a turntable and a pedal to 14-key doubles and a catch mode. The cabinet studied here has a Pentium Dual-Core E5800 (3.2 GHz), 2 GB of RAM, a GeForce GT 520 and Windows XP Professional 32-bit. On that system the game process has 2 GB of address space.

On home and cabinet installations alike, the game is started by the community launcher 2EZConfig, which emulates the I/O board, offers configuration patches and can load additional user DLLs into the game. Our extension is such a DLL. It integrates with the game in memory and leaves every game file on disk untouched.

2.2 BMS

A BMS song is a folder: one or more plain-text charts (`.bms`, `.bme`, `.bml`) and their sounds and images. A chart assigns sounds to positions within measures on numbered channels, one per lane, with separate channels for background music, tempo changes, stops and long notes. Charts declare a difficulty level (`#PLAYLEVEL`) but no text encoding. The library used here has 23,587 songs in 106 folders, gathered from many community packs whose conventions differ.

3. Methodology

No single source of evidence proved reliable on its own, so every important claim was confirmed by at least two of the following.

- **Static decompilation.** Four decompilers (a Ghidra headless export of 2,014 functions, IDA/Hex-Rays, Snowman and Rec Studio) disagree on exactly the code that is hard: compiler-specific calling conventions, stack-cookie prologues, jump tables and SSE floating point. Disagreement marks the instructions worth reading by hand.
- **A byte-matching decompilation.** A separate private project, EZ2DECOMP, reconstructs source that recompiles with the original compiler into byte-identical functions, verified per function with `reccmp` [1]. It identified that compiler from the Rich header (Visual C++ 7.1, build 3077) and reconstructs every referenced function; 1,351 of 1,563 compared entities match byte for byte. A matched function is the original logic, not an interpretation of it.
- **Run-time observation.** Our DLL logged arguments, structures and timings from inside the running game. Early versions did nothing else. "Marker builds" settled questions of identity by writing distinct markers into candidate arrays and observing which one appeared on screen.
- **Library-wide sweeps.** Every format parser was run over everything the game ships and required to land exactly on the end of every file. Every rule in Section 4.3 that was wrong in a first draft was corrected by

a sweep, not by reasoning.

- **Measurement on the target.** All performance claims come from logs written on the cabinet, with a millisecond clock on every line and a per-frame breakdown of where the time went.

The strongest check was a **differential trace**. The game's judgement sink was observed in the running game; each call was recorded with its inputs, its call site and the state before and after; and the record was replayed through an independent reimplementation of the scoring rules. A first live capture of one stage (552 events) and a two-player capture (226 events) both replayed with zero divergences. The trace was needed because reading had failed: in the scoring code alone, four readings that looked right were wrong, and only one of them had been caught by a mechanical check.

Handling rules. No bytes of the original executable, no assets and no key tables are reproduced in this report or in the project's documentation. Addresses, structure layouts and descriptions of behaviour are analysis. The retail executable is licence-protected (Section 4.1); we describe that protection only to explain why all analysis targets an unpacked image, and we give no procedure for obtaining one.

4. Characterising the game

4.1 The binary

The retail executable is protected: a dropper at the entry point starts a licence-checking loader, and the game's code section is encrypted with AES-128 in ECB mode under a key derived at run time from the licence. The key is not stored in the file. All analysis therefore uses an unpacked image of the same build.

That image can be validated without the key. ECB maps every 16-byte block to a fixed ciphertext block, so pairing the image's code blocks with the retail file's must give a consistent one-to-one codebook. For our image the codebook is a perfect bijection over 31,983 distinct blocks, with zero conflicts in either direction; the data section is identical, and the read-only section differs only in its import table, which the protector rebuilds at run time.

The image has a fixed base address with no relocations, so every function and global keeps the same address on every run. It also has a **53.7 MB static data section**: most game state lives in fixed global tables (input, songs, categories, chart queues), and most of the game's limits are simply the sizes of those tables. The compiler adds stack cookies only to functions with character buffers (275 of them), and links the C runtime statically.

4.2 The engine

A screen driver runs one *director* per screen: title, mode select, song select, three gameplay directors (key modes, scratch modes, catch mode), result and ranking. Engine services provide graphics, sound, input and configuration:

- **Graphics.** The renderer draws pre-transformed quads, blends through two render states, and offers sixteen fixed-function multitexture recipes, the same sixteen presets the authoring tool exposes.
- **Textures.** Textures use a black colour key rather than alpha (43,819 of 62,201 shipped textures rely on it). They are cached by file name and released only when a screen ends.
- **Sound.** Sound is one DirectSound buffer per voice, two voices per sample, driven by a pump thread that wakes every millisecond. That thread, not the frame loop, advances the song clock.
- **Text.** The game ships two bitmap fonts: ASCII, and the 2,350 Hangul syllables of KS X 1001. Song titles are not text at all: every title is a pre-rendered 256×32 picture named after the song.

4.3 File formats

Table 1 summarises the sweeps. Most formats descend from 1990s tools and still carry their original four-character tags. Community documentation [2] described the containers; the sweeps added the rules below.

Format	Content	Files	Parsed	Notable rule
`.ez`	charts (v7/v8)	12,361	12,359	64 tracks; lengths biased by 6; the v7 widening shifts every field
`.ezi`	keysound indexes	12,359	12,359	mode-2 lines carry a second name; 4,703 references point into other songs
`.ssf`	samples	178,474	178,470	no magic; header validated by its two redundant fields
`.str`	animated scenes	10,593	10,593	three variants; rotation is a table index
`.scr`	scene timelines	314	311	planes play concurrently; times in 60 Hz frames
`.ini`	judgement and gauge	12,342	12,342	values read by position; 327 window sets
`.abm`	textures	62,201	62,201	XOR-masked BMP header; 16-bit is 1:5:5:5

Table 1. Whole-library parse results. The few failures are corrupt files in the shipped game, identified by independent checks (for example, a healthy chart contains its track tag exactly 64 times).

Several rules are silent when wrong:

- **The `.ez` v7 widening.** v7 widened the keysound index by one byte. Reading a v8 chart at the older offsets turns every note's length of 6 into 1,536: every note becomes an endless hold, yet the chart still parses and looks plausible.
- **Two spare bytes in `.abm`.** In 35,376 textures two spare bytes lie at the end of the file, not the start. Placing them wrongly shifts every pixel, yet the result is still a recognisable picture; a greyscale mask decoded both ways (100% versus 93% grey) settled it.
- **Lane order is data.** A mode's lane order comes from a plain-text descriptor read in file order, not from code.
- **Scene times.** The animated-scene timelines express time in 60 Hz frames. Across 3,251 entry pairs, the gap the timeline leaves for a scene equals that scene's own length (median ratio 1.00); read as milliseconds, every scene plays 16.7 times too slowly.
- **File obfuscation.** The game's data files are obfuscated with a 512-byte additive keystream and a fixed block permutation, the same for every file of a kind.

4.4 Timing and judgement

The clock. The judge's clock counts chart ticks. The chart loader rescales the file's 48 ticks per beat by four, so a tick is 1/192 of a beat:

```
1 tick = 312.5 / BPM  milliseconds
```

The press time. A press is detected once per frame. Its time is synthesised when the frame processes it, as the current tick minus a fixed five-tick compensation, about the mean delay of frame sampling.

The windows. Each chart ships its own windows in its `.ini`, and the engine widens them by three ticks before use. The windows are therefore **beat-relative**: the same file is three times stricter in milliseconds at 200 BPM than at 66 BPM (Figure 1, Table 2).

Grades. A press scores KOOL, COOL or GOOD. A press outside GOOD but inside the miss window is a FAIL, which drains the gauge but does not break the combo. Only a note nobody pressed is a MISS. Community documentation had read the window values as milliseconds.

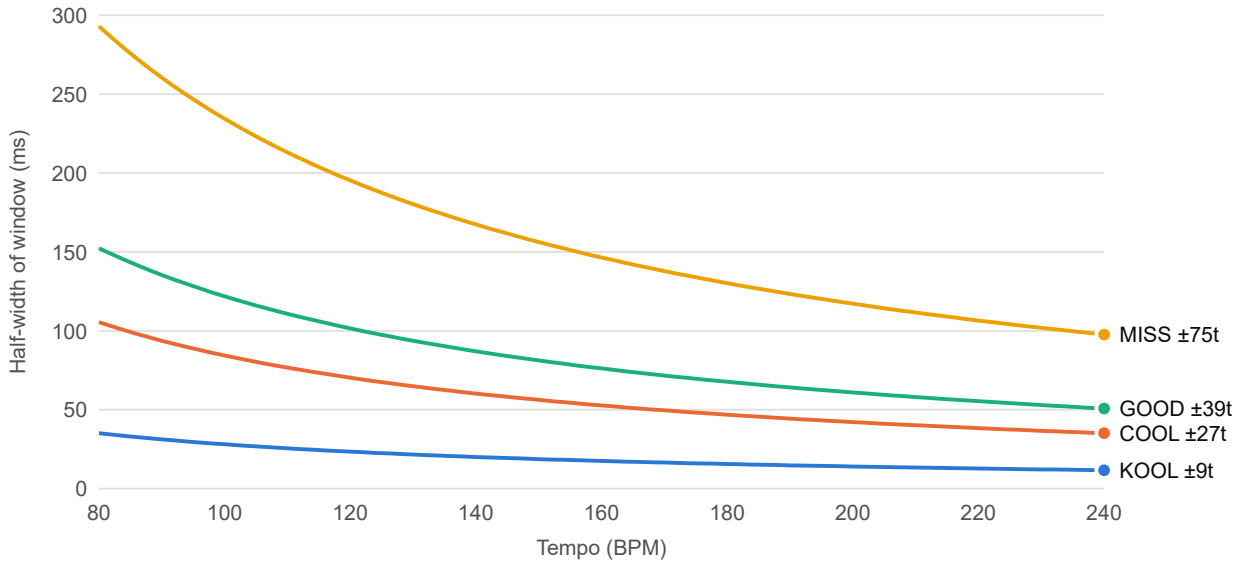


Figure 1. Half-width of each judgement window in milliseconds as a function of tempo, for the default windows after the engine's +3-tick widening (KOOL ±9, COOL ±27, GOOD ±39, MISS ±75 ticks). Each curve is `ticks × 312.5 / BPM`; Table 2 lists the same values.

Window	90 BPM	120 BPM	150 BPM	180 BPM	200 BPM	240 BPM
KOOL (±9 ticks)	±31	±23	±19	±16	±14	±12
COOL (±27 ticks)	±94	±70	±56	±47	±42	±35
GOOD (±39 ticks)	±135	±102	±81	±68	±61	±51
MISS (±75 ticks)	±260	±195	±156	±130	±117	±98
1 tick (ms)	3.47	2.60	2.08	1.74	1.56	1.30

Table 2. The data of Figure 1, in milliseconds. Shipped charts often override the defaults; the most common set (6/24/50/70) covers only 45% of charts.

Hold notes. A hold is graded at its head and at its tail, by the same function against the same windows, but it is counted once: neither end emits the ordinary judgement. The engine computes the average of the two grades and then discards it. A held note pays out in instalments while it is held, on a step chosen by a "kind" byte in the note record. A trace of one chart with 31 holds shows exactly 309 instalment payments, which equals `Σ(length/12 − 1)`; its 551 judged notes are 242 heads plus those 309 instalments. For one kind, the function that counts notes towards the maximum and the function that pays instalments disagree, which would make a perfect score unreachable. No shipped chart contains such a note.

Scoring. In the key modes each note is worth a flat 300 / 150 / 41 points for KOOL / COOL / GOOD, against a maximum of 300 per note. Ez2Catch accumulates a logarithmic combo term instead, with truncation (not rounding) and a single-precision constant; both details change totals from the third note onwards.

The gauge. The gauge rates are read from the `.ini` by position, so the rate named `Fail` is what an unpressed note costs, and the rate named `Miss` is what a press in the outermost window costs. Once the gauge reaches zero it stays there.

5. Bringing BMS into the game

5.1 Song select at 55 times its design size

Song select was designed around the shipped list:

- every category's rows are gathered into a 436-entry table, so 436 is the per-category limit;
- each id is found by a linear, case-insensitive search of the whole song list;
- every title is a picture that must exist before it can be shown;
- the wheel has 47 category slots, of which only 16 hold songs of their own (the game's versions); the rest are views of the same songs sorted by level and alphabet.

The extension enlarges the list structures and the category machinery to 23,587 songs in 106 categories (up to 872 songs each). It keeps the 19 native categories that present the game's own songs, renders song-title pictures on demand on a background thread, and keeps graphics resources inside a fixed budget (Section 6). The original semantics are kept wherever they can be observed. In particular, song select keeps publishing counts exactly as the original does, because other parts of the program depend on them.

5.2 A finer grid

Shipped charts use 192 ticks per measure. Measured over 45,707 notes from 80 BMS charts, the dominant BMS subdivision is 1/384 of a measure, so on a 192 grid **24% of notes fall between two ticks and are displaced by a mean of 4.17 ms**, almost always by a full half-tick. At the tightest window (about ±14–23 ms, Table 2) that is a large share of the timing budget, and players reported that converted charts felt harder to time than native charts of the same level. The engine honours a chart's declared resolution, so the converter writes 384 ticks per measure: the dominant case becomes exact and the worst case for every other subdivision is halved. Positions are converted through an integer grid of 57,657,600 per measure, which every BMS subdivision in use divides exactly.

5.3 Repairing long notes

One of BMS's two long-note encodings marks a hold with a head event and a tail event on a dedicated channel, and players pair them by alternation. A single missing or extra event inverts every hold after it on that lane; an audit found this in hundreds of charts. Because a head and its tail are always consecutive events on their lane, we pair each lane with a small dynamic programme: each event is either paired with the next or played alone as a tap, with a penalty for a pair that would swallow an ordinary note (1,000) and for a lone event (1,500). A well-formed lane decodes exactly as alternation would, and a broken lane loses one event instead of everything after it.

5.4 Folding onto five keys and generating other modes

Packs ship 7-key, pop'n and double charts alongside 5-key ones. For the 5-key mode, every chart is folded onto five keys: lanes keep their left-to-right order, larger layouts map proportionally, and notes landing on a busy key move to the nearest free one. Notes that fit nowhere keep sounding as background. Native 5-key charts contain no scratch notes at all (0 of 168,643), so scratch lanes in 5-key charts take the free key nearest key 1. Charts written for another layout are played as written in the matching mode: 503 songs carry 7-key charts, 97 carry 5+5 doubles and 51 carry 7-key doubles.

For the remaining modes, **AutoChart** generates charts from the 5-key source, using statistics learned at start-up from up to 192 of the game's own charts per mode:

- which physical tracks each mode really uses, ranked by share of notes;

- the share of scratch, pedal and second-side notes;
- a 16-position histogram of where in the beat scratch and pedal notes fall;
- the mean gaps between consecutive scratch notes and between pedal notes;
- how often those notes take part in chords and holds.

The learned track usage corrected several hand-written layouts: one mode uses five lanes, not four, and two use a second side. Generation is deterministic. It chooses scratch and pedal ticks from the learned distributions and from the keysound's name, redistributes the remaining notes over the mode's lanes, locks a lane until its hold ends, and sends leftover simultaneous sounds to background tracks. The library expands to 212,283 catalogue entries.

5.5 Rating charts from their notes

Declared levels saturate at the top: **46% of songs called their hardest chart 18**, and within that level the busiest two seconds ranged from 13 to 83 notes per second. We rate each chart from its notes:

```
score = 0.8 × peak + 0.2 × (2.6 × average)
```

- **peak** is the busiest two seconds and **average** the mean density, in notes per second;
- the factor 2.6 aligns the two (their ratio holds at 2.58–2.62 from level 5 to 18);
- the peak ranks songs well across the library, and the average separates a song's own charts, which usually share the same climax.

The score maps onto levels 1–20 through a monotone curve fitted to the mean peak of 1,379 charts that declared levels 2–17. Level 18 is excluded from the fit, because it is where packs put everything they consider hard. One level is added for heavy jacks (repeated notes in one lane), whose rate climbs from 0% at level 2 to 15.5% at level 18. Over 400 songs, the hardest-chart level that piled 45.8% of songs onto 18 now spreads over 14 levels, with 14.0% in the fullest one; a peak-only rating left 45 songs with two or more charts on the same level, the blend 23. At start-up, 84,210 of 84,291 charts are rated this way, and the rest keep their declared level.

6. Performance on the original hardware

The cabinet's start-up timeline (Figure 2) shows where 26.9 seconds went before the game window appeared. The two longest steps, verifying the catalogue order and sizing song select (7.5 s and 7.2 s), spent most of their time building sorted per-mode copies of a 212,283-entry catalogue. The sort's comparator located category ranks by a linear string search, twice per comparison, and moved whole song records on every swap. Sorting (rank, index) pairs with ranks from a hash map, and copying each record once, gives the identical order (checked on 300 random catalogues) at a fraction of the cost.

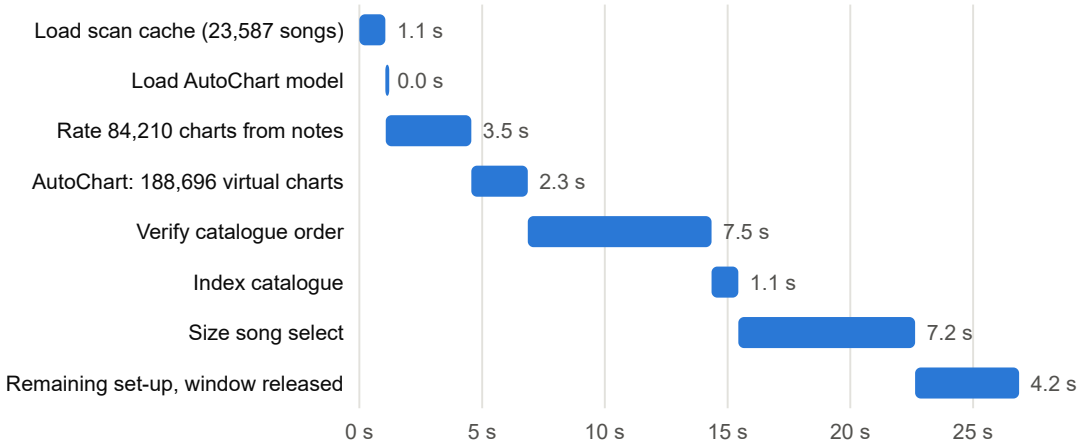


Figure 2. Start-up timeline from the cabinet's log of 2026-10-19 (version 36.0.0). Bars are the consecutive steps between log timestamps; labels give each step's duration.

Two freezes in song select were found only on the cabinet:

- **Category changes (0.7–1.7 s).** Every category change produced a slow frame of 0.7–1.7 s, independent of the category's size. On every change, a preparation step rebuilt the full sorted catalogue only to find that the custom songs were already in place. Checking that first removed the cost from category changes entirely.
- **Resting on a song (0.4–0.6 s).** The game starts a song's audio preview half a second after the cursor stops. For a custom song the preview had to be cut from an MP3 first, on the game's own thread, which froze the game for 0.4–0.6 s, 86 times in one session, and also stopped the turntable being read. The cut now runs in the background from the moment the cursor settles, and the game's preview countdown is held until the file is ready.

Graphics resources needed a budget of their own. The game frees no texture while a screen is alive, and every texture costs one of the 10,000 GDI objects Windows grants a process:

- loading whole categories exhausted that budget after about 25 categories;
- loading only a window around the cursor still grew the process from 840 MB to 2,551 MB over 110 categories;
- at about 251 KB of surface per texture, walking every category would need 1,979 MB, almost all of XP's address space.

A fixed ring of reusable textures for song titles and discs makes browsing cost constant; sessions now sit near 2,200 GDI objects.

7. Discussion

Reading fails; mechanical checks do not. The most expensive errors of the project were confident readings of correct-looking disassembly. The differential trace, parse-to-end sweeps and checks by redundant quantities caught what re-reading missed. We recommend building such checks before relying on any reading of scoring or timing code.

The tail defines the format. Every format rule in Section 4.3 that was wrong in its first draft held for the sample files and failed somewhere in the long tail of the library. A format parser is not done until it has seen everything the game ships.

Measure on the target. Both freezes of Section 6 were invisible on a development PC and cost a second each on the cabinet. Legacy hardware is a different machine, not a slower one: work that is negligible on one becomes the dominant cost on the other.

Keep the original's semantics, including the odd ones. Several of the game's behaviours look like mistakes: the crossed gauge names, a discarded average, counts that disagree. Changing them changes the game, and in one case an attempt to "correct" what song select counts broke an unrelated part of the program. A faithful extension reproduces them.

Limitations. The work was validated on one cabinet. The differential traces cover the key modes and one or two players, not Ez2Catch or every option. CV2Mix has no single lane layout and is generated rather than characterised. The AutoChart generator is statistical: it reproduces how native charts use scratch and pedal, not how a human charter would arrange a song.

Ethics and legality. The game is a commercial product whose rights holder is active, with a current title on another platform. The work is private: it distributes no part of the game, no keys and no assets; it modifies no game file; and it is described here for interoperability and preservation. It relies on an unpacked executable that is already part of the community's installations, and it documents the licence protection only at the level of analysis. Anyone building on it should take legal advice before publishing.

8. Related work

- **Matching decompilation.** Recent projects such as the LEGO Island decompilation and its `reccmp` tool [3] reconstruct game source that recompiles byte-identically. EZ2DECOMP [1] applies that method to this game; our work consumes its results and adds run-time evidence and whole-library sweeps.
- **Engine reimplementation.** ScummVM and OpenMW reimplement game engines and ask users to supply the original data. Our work instead extends the original program in place, which keeps its exact behaviour at the cost of working within its limits.
- **BMS players.** Community BMS players such as Lunatic Rave 2 and beatoraja [4] play BMS natively with their own timing models. Our aim was the opposite: the original game's timing and scoring applied to BMS charts.
- **Community tools.** freem's format notes and tools [2] and the 2EZConfig launcher [5] are the foundation every modern EZ2 installation stands on.

9. Conclusion

A dormant arcade game can be given a living library without modifying it and without leaving its original hardware. Doing so required a characterisation deep enough to reproduce its judgement and scoring exactly, a conversion that respects both BMS conventions and the engine's constraints, and performance work grounded in measurements on the cabinet itself. The method behind the results is general: combine independent evidence, verify mechanically, sweep the whole data set, and measure on the target machine.

References

1. EZ2DECOMP: a byte-matching decompilation of EZ2AC. Private project, 2026.
2. freem. *ez2stuff*: EZ2DJ/EZ2AC tools and file-format notes (`formats.md`).
3. isledecomp contributors. *LEGO Island decompilation* and *reccmp*. https://github.com/isledecomp/isle
4. exch. *beatoraja*, a cross-platform BMS player. https://github.com/exch-bms2/beatoraja
5. kasaski. *2EZConfig V2* (GPL-3.0), a rewrite of ben-rnd's 2EZConfig. https://github.com/ben-rnd/2EZConfig

6. lieff. *minimp3*. https://github.com/lieff/minimp3 · S. Barrett. *stb_vorbis*. https://github.com/nothings/stb
7. Microsoft. *DirectX 7.0 SDK documentation*: Direct3D texture stages and DirectDraw surfaces, 1999.

This report summarises the project's research documentation (`docs/RESEARCH.md` and `docs/research/`), where each claim is given with its addresses and evidence.